CHIANG MAI UNIVERSITY

Bachelor of Science (Software Engineering)

College of Arts, Media, and Technology

SE 233 Advanced Programming

Term project II

1) Preface

- Every year, the second term project for SE233 requires students to replicate a simple game, encompassing the skills practiced throughout the course. This includes event handling, exception management, multithreading, logging, debugging, and building management.
- For this year, we have selected a roguelike-style version of the classic Arkanoid/brick-breaker game. Students are required to implement the core brick-breaking gameplay and add a roguelike upgrade system that changes the player's abilities as the game progresses.
- A full replication of an existing Arkanoid game isn't necessary. Students can choose the desired features based on the scoring rubric provided in Section 4. Additionally, the game's GUI or game balance don't need to be perfectly matched with an original Arkanoid game.
- The Arkanoid gameplay sample
  i) https://www.youtube.com/watch?v=Th-Z6QQ5AOQ
  ii) https://www.youtube.com/watch?v=zb6ik3hk1xE

2) Setup

- **Pairing Up**: You are to work in pairs. Pairs can be formed across sections, meaning two students can be from section 701, from 702, or one from each section (701 and 702).
- **Evaluation**: An onsite demonstration is required for grading. Both lecturers will assess your project based on the scoring rubrics provided in Section 4 of these instructions. Your final score will be the average of the scores given by the two lecturers.
- **Key Dates**:
  - **Assignment Date**: 5 October 2026
  - **Code Submission Deadline**: 2 November 2026
  - **Onsite Demonstration**: 3 November 2026

3) Use Cases

3.1 Start Game

- The player starts a new game from a simple start menu.
- The game initializes the paddle, ball, bricks, score, and lives.

3.2 Play Stage

- The game is designed for one player and uses a 2D play area.
- The player controls a paddle using keyboard buttons.
- The ball moves continuously and bounces from the play area's boundaries and paddle.
- When the ball hits a brick, the brick is damaged or destroyed according to its type.
- The player earns points when bricks are destroyed.

3.3 Complete Stage and Choose Upgrade

- When the required bricks are cleared, the current stage is completed.
- The game pauses and presents several randomly selected upgrade choices.
- The player selects one upgrade before continuing.
- The selected upgrade affects gameplay in the following stage and/or the remainder of the current run.
- Students must implement at least three different upgrade types.

3.4 Lose Life / Game Over

- When the player fails to return the ball, the player loses a life.
- The player initially has 3 lives, or an equivalent clearly defined starting amount.
- When all lives are lost, the game ends.
- The game displays appropriate completion of feedback for winning or losing.

3.5 Gameplay and Design

- Students are free to design the arrangement and types of bricks.
- Students are free to design the stage structure and difficulty progression.
- Students may implement different brick types, such as normal, strong, bonus, or special bricks.
- Students may design their own scoring system, provided that it is clear and consistently implemented.
- Students may provide appropriate backgrounds and visual effects.

3.6 Roguelike Upgrades

Possible upgrades include:

- Increase paddle size.
- Increase paddle movement speed.
- Increase or decrease ball speed.
- Add an additional ball.

- Gain an additional life.
- Allow the ball to pass through bricks.
- Increase score gained from destroying bricks.

Students are free to design their own upgrades. The upgrade system should have a meaningful effect on gameplay rather than only changing the appearance.

Also, students may design items to receive during gameplay, such as, allow laser shooting for 30 seconds.

3.7 feedback

- Display the score and important game state.
- Provide visual feedback when bricks are hit or destroyed.
- Provide feedback when a stage is completed, an upgrade is selected, a life is lost, or the game ends.

3.8 Sound

- No background music (BGM) or sound effects are not mandatory. However, students may implement sound effects, for example, ball hitting a paddle/brick, brick destruction, upgrade selection, or losing a life.
- Sprite sheets can be retrieved from https://www.spriters-resource.com/arcade/arkanoid or other preferred platforms.

4) Rubrics

4.1 For 2-member group

Item	Get 1 point	Get 2 points	Get 3 points
Functionalities	It can be played as a Arkanoid-like game, and all the requirements are fulfilled.	Multiple stages are implemented and a functional roguelike upgrade-selection system is included.	
Sprite sheet	Move and ball bouncing actions are separated.	Plus, a special attack is implemented, such as, allow dual laser for a specific of time (Depends on your design).	
OOP	There are at least 2 meaningful classes for game objects or responsibilities.	There are at least 3 meaningful classes for game objects and other necessary responsibilities.	
Display score	Show score when a brick is hit/destroyed and the score number changes.		
Additional animation	Basic visual feedback is shown when a brick is hit or destroyed.		
Exception handling	You throw all the exceptions of the same types and handle each of them in one place.	.	

Logging	Log important game events.	Log game/player actions and scoring at different log levels.	
Testing	Unit tests for important game logic such as ball/brick behavior or scoring.	Unit tests for game actions, scoring, lives, and/or upgrade behavior.	
Build management	A Maven project where its life cycle includes compiling, running, test, and making an executable jar.		
Additional features **(Subjective)**	Add one more stunning feature that does not exist in the original version.		

4.2 For1 or 3-member group

Item	Get 1 point	Get 2 points	Get 3 points
Functionalities		It can be played as a Arkanoid-like game, and all the requirements are fulfilled. Multiple stages are implemented, and a functional roguelike upgrade-selection system is included.	
Sprite sheet		Move and ball bouncing actions are separated. Plus, a special attack is implemented, such as, allow dual laser for a specific of time (Depends on your design).	
OOP		There are at least 3 classes of paddle, bricks, and other necessary classes.	There are more than 3 classes, and they show implementation of strongly OOP concepts such as class relations.
Display score	Show score when a brick is hit/destroyed and the score number changes.		
Additional animation	Basic visual feedback is shown when a brick is hit or destroyed.		

Exception handling	You throw all the exceptions of the same types and handle each of them in one place. Also, define your own exception class.		
Logging	Log game/player actions and scoring at different log levels	.	
Testing	Unit tests for important game logic such as ball/brick behavior or scoring.	Unit tests for game actions, scoring, lives, and/or upgrade behavior.	
Build management	A Maven project where its life cycle includes compiling, running, test, and making an executable jar.		
Additional features **(Subjective)**	Add one more stunning feature that does not exist in the original version.		